Informe de Laboratorio: Estructura de Computadores

Nombre del Estudiante: Oscar Yamid Lozano Tamy **Fecha:** 02/03/2025
Asignatura: Estructura de Computadores

Enlace del repositorio en GitHub: https://github.com/OlozanoTamy/Optimizaci-n-en-Procesadores-Segmentados

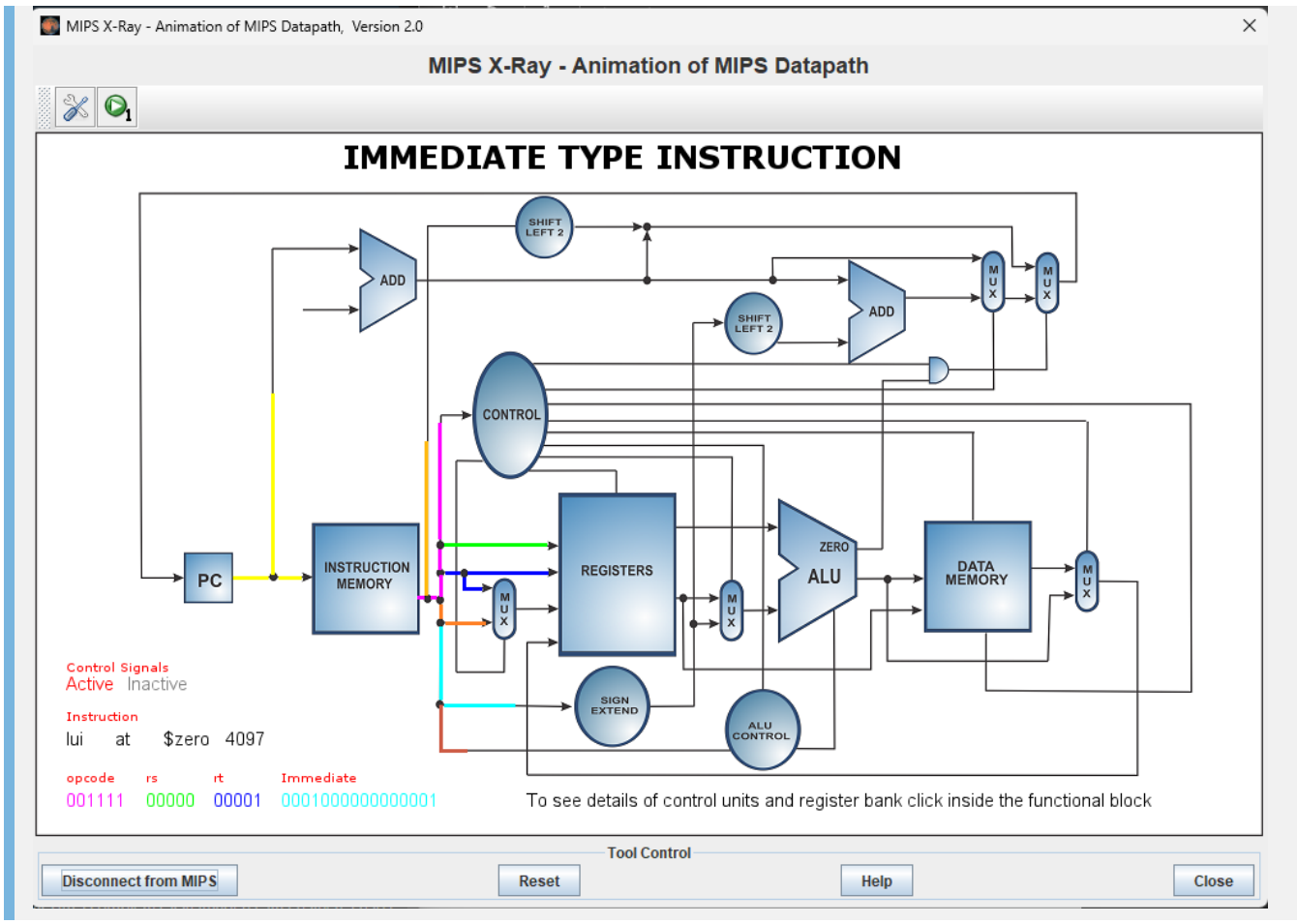
---

1. Análisis del Código Base

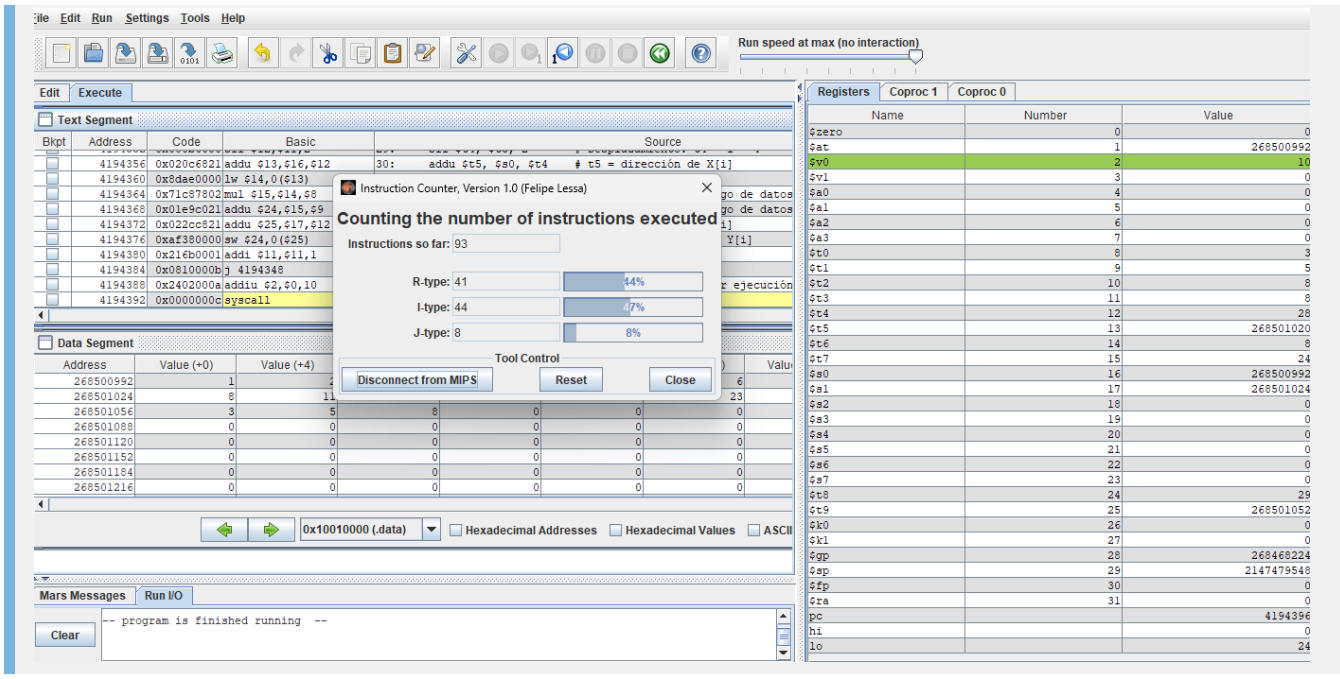
1.1. Evidencia de Ejecución

Adjunte aquí las capturas de pantalla de la ejecución del `programa_base.asm` utilizando las siguientes herramientas de MARS:

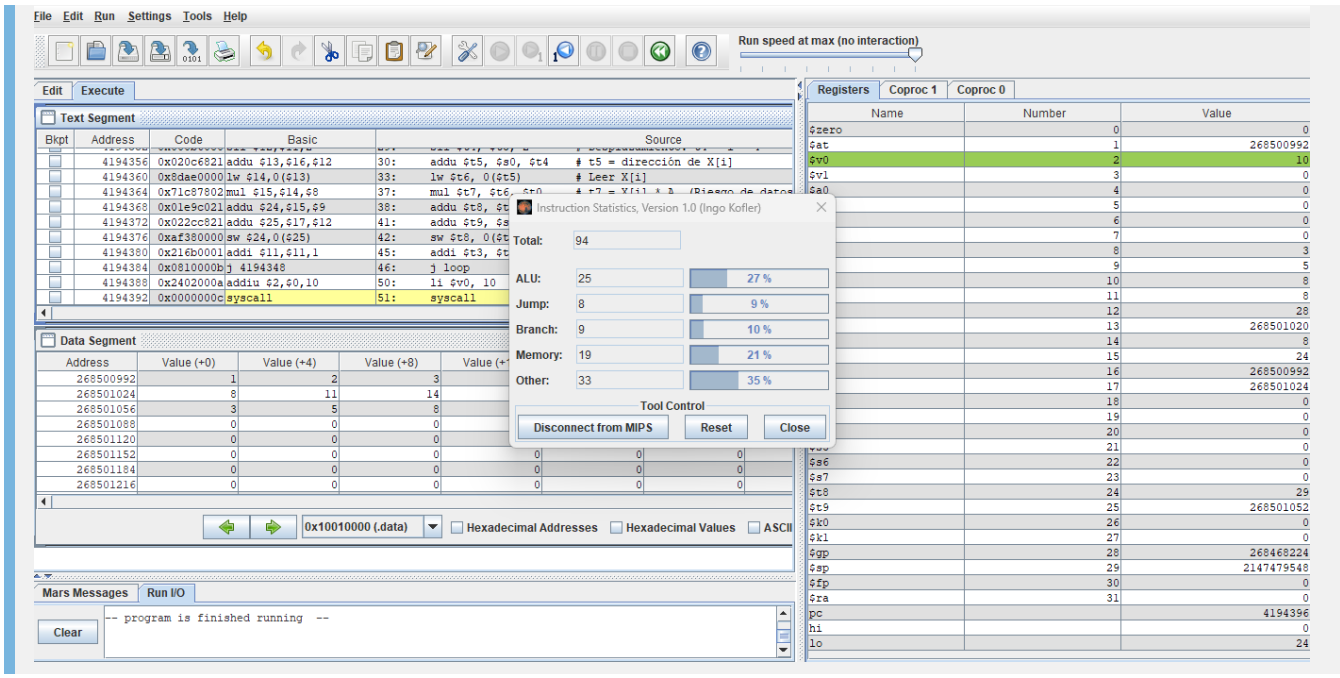
- **MIPS X-Ray** (Ventana con el Datapath animado).



- **Instruction Counter** (Contador de instrucciones totales).



- **Instruction Statistics** (Desglose por tipo de instrucción).



1.2. Identificación de Riesgos (Hazards)

Completa la siguiente tabla identificando las instrucciones que causan paradas en el pipeline:

Instrucción Causante	Instrucción Afectada	Tipo de Riesgo (Load-Use, etc.)	Ciclos de Parada
`lw $t6, 0($t5)`	`mul $t7, $t6, $t0`	Load-Use	1

1.2. Estadísticas y Análisis Teórico

Dado que MARS es un simulador funcional, el número de instrucciones ejecutadas será igual en ambas versiones. Sin embargo, en un procesador real, el tiempo de ejecución (ciclos) varía. Completa la siguiente tabla de análisis teórico:

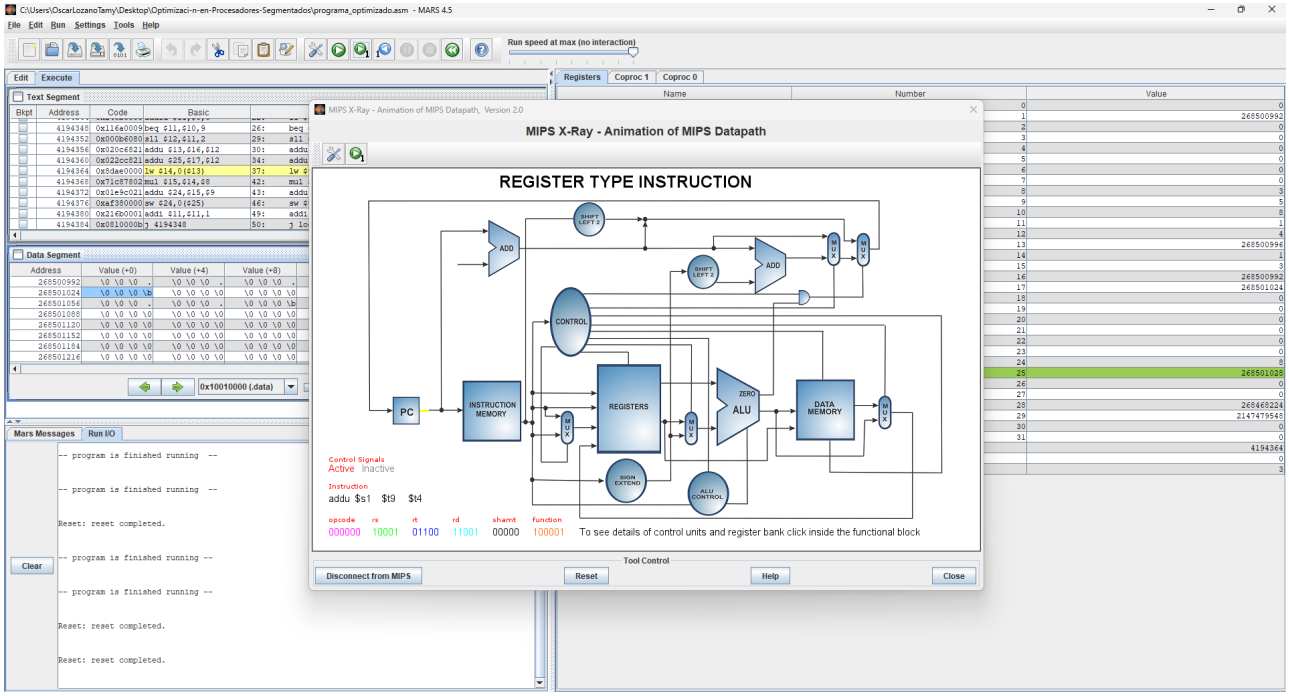
Métrica	Código Base	Código Optimizado
Instrucciones Totales (según MARS)	93	93
Stalls (Paradas) por iteración	1	0
Total de Stalls (8 iteraciones)	8	0
Ciclos Totales Estimados (Inst + Stalls)	101	93
CPI Estimado (Ciclos / Inst)	1.09	1.00

2. Optimización Propuesta

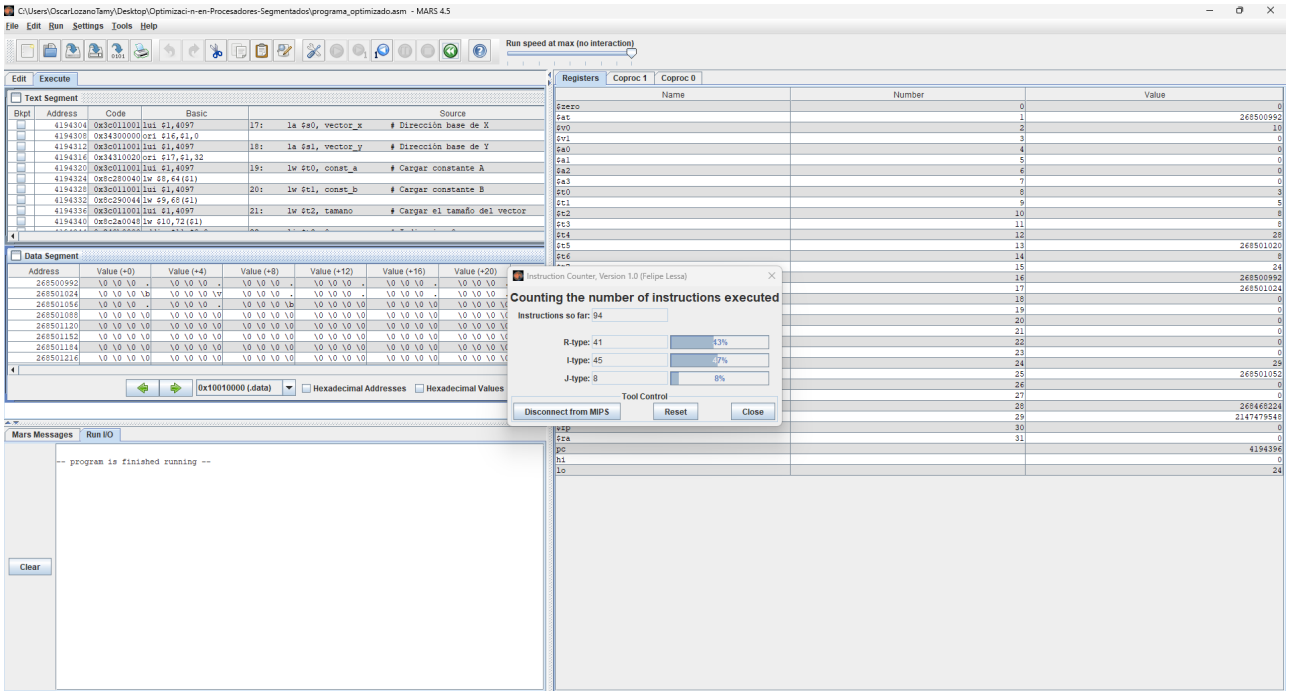
2.1. Evidencia de Ejecución (Código Optimizado)

Adjunte aquí las capturas de pantalla de la ejecución del `programa_optimizado.asm` utilizando las mismas herramientas que en el punto 1.1:

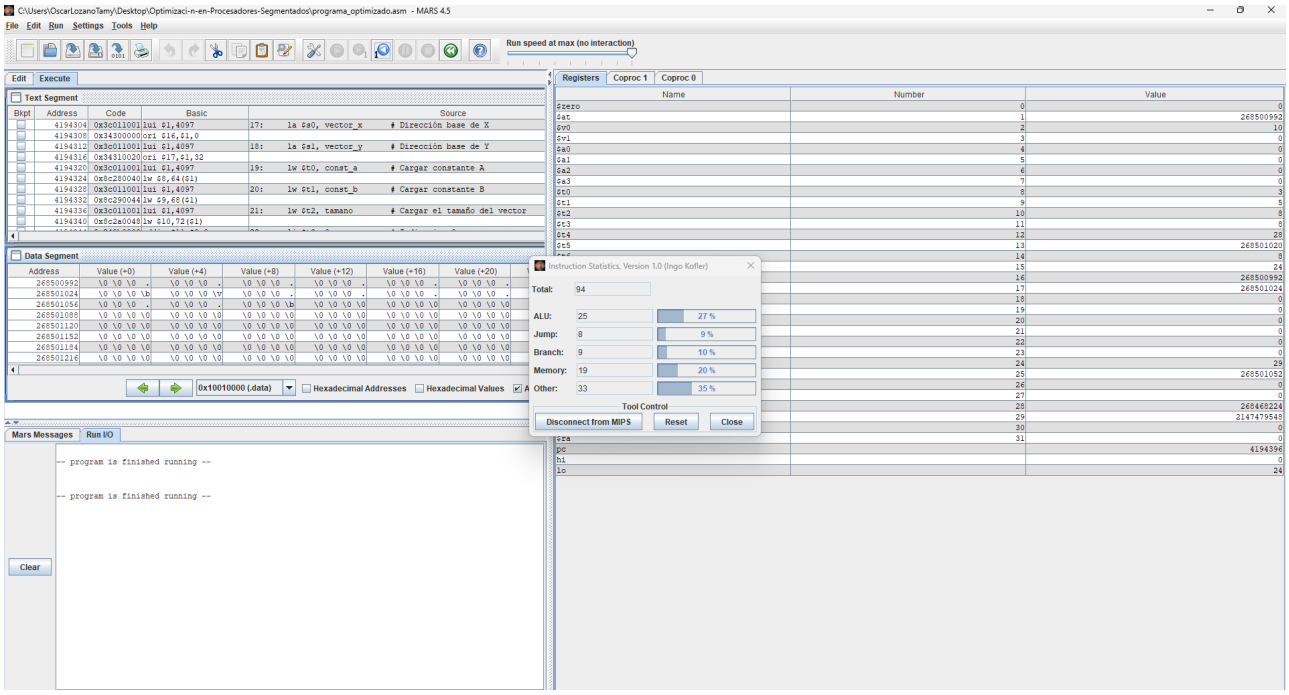
- **MIPS X-Ray**.



- **Instruction Counter**.



- **Instruction Statistics**.



2.2. Código Optimizado

Pega aquí el fragmento de tu bucle `loop` reordenado:

```
# loop:
    # --- Condición de salida ---
    beq $t3, $t2, fin      # Si i == tamano, salir del bucle

    # --- Cálculo de dirección de memoria ---
    sll $t4, $t3, 2        # Desplazamiento: t4 = i * 4
    addu $t5, $s0, $t4     # t5 = dirección de X[i]

    # --- OPTIMIZACIÓN: calcular dirección de Y[i] mientras se espera el lw ---
```

```
    # Esta instrucción es INDEPENDIENTE y oculta el ciclo del load-use hazard
    addu $t9, $s1, $t4    # t9 = dirección de Y[i]

    # --- Carga de dato ---
    lw $t6, 0($t5)        # Leer X[i]
    # ANTES: Aquí habría un stall porque 'mul' usa $t6 inmediatamente
    # AHORA: El ciclo se ocultó con el cálculo de t9 arriba

    # --- Operación aritmética ---
    mul $t7, $t6, $t0     # t7 = X[i] * A
    addu $t8, $t7, $t1    # t8 = t7 + B

    # --- Almacenamiento de resultado ---
    sw $t8, 0($t9)        # Guardar resultado en Y[i]

    # --- Incremento y salto ---
    addi $t3, $t3, 1      # i = i + 1
    j loop

fin:
    # --- Finalización del programa ---
    li $v0, 10            # Syscall para terminar ejecución
    syscall
```

2.2. Justificación Técnica de la Mejora

Explica qué instrucción moviste y por qué colocarla entre el `lw` y el `mul` elimina el riesgo de datos:

> Colocamos la instrucción `addu $t9, $s1, $t4` que corresponde a la variable donde realizamos el cálculo de la dirección de `Y[i]`, después de calcular el desplazamiento `t4` y antes del `mul` que usa el dato cargado por el comando `lw`. El calculo de esta variable `addu` es independiente del valor leído desde memoria, por lo que puede ejecutarse mientras la carga (`lw $t6, 0($t5)`) se completa con el acceso a memoria.

> El hazard(Riesgo) que se identifico es un "load-use", es decir, la instrucción siguiente usa el registro que produce el `lw`. Si no se hace nada, el procesador necesita insertar un ciclo de espera (stall,que es lo que se requiere evitar en el codigo optimizado) para que el dato esté disponible. Al reordenar con una instrucción independiente entre `lw` y `mul`, ocupamos ese ciclo útil con el calculo de una variable (calcular la dirección de `Y[i]`), eliminando así el ciclo de parada visible.

---

3. Comparativa de Resultados

Métrica	Código Base	Código Optimizado	Mejora (%)
Ciclos Totales	101	93	7.92%
Stalls (Paradas)	8	0	100%
CPI	1.09	1.00	8.26%

---

4. Conclusiones

¿Qué impacto tiene la segmentación en el diseño de software de bajo nivel? ¿Es siempre posible eliminar todas las paradas? La segmentación (pipeline) mejora el rendimiento en general del diseño de software,por que permite aprovechar los espacios en el procesador y ahorrar tiempo, pero introduce nuevos retos a la hora de escribir código de bajo nivel debido a las dependencias entre instrucciones. Mediante las técnicas sencillas de reordenamiento de instrucciones (como la que se aplico en este laboratorio) se pueden ocultar muchos stalls sin cambiar el funcionamiento general del programa, lo que reduce el número de ciclos desperdiciados y mejora el CPI.

Sin embargo en mi criterio, no siempre es posible eliminar todas las paradas solo con reordenamiento: algunas dependencias son realmente inevitables (por ejemplo, una operación suma que necesita el resultado previo y que no se puede posponer)

En conclusion, la segmentación requiere una rigurosa atención al orden de las instrucciones para obtener un buen desempeño; con pequeñas reordenaciones como la mostrada se consiguen mejoras reales en el tiempo de ejecución sin aumentar el número de instrucciones ejecutadas.